

Database Management – SQL & PL/SQL

Theme: Personal Expense Tracker DB

Section A: Concept Application

1. How do relational databases help maintain accuracy in expense records?

Relational databases maintain accuracy through a well-defined structure:

- Primary Keys ensure each record is unique — no duplicate expense entries.
- Foreign Keys enforce that expenses only reference valid users and categories.
- Data Types (e.g., DECIMAL for amount) prevent invalid values from being stored.
- ACID Transactions ensure multi-step operations either fully complete or fully roll back.
- Normalization removes redundant data — user info is stored once and referenced by ID.

2. Why are constraints important in personal finance data?

Constraints act as rules that protect the integrity of financial data at the database level:

- NOT NULL: Ensures fields like amount or expense_date are always filled.
- PRIMARY KEY: Uniquely identifies each record, essential for audit and tracking.
- FOREIGN KEY: Links expenses to real users and categories, preventing invalid references.
- CHECK: Restricts values, e.g., CHECK(amount > 0) blocks negative amounts.
- UNIQUE: Prevents two users from having the same email address.

Without constraints, invalid data can enter silently, causing incorrect financial calculations.

3. How does GROUP BY help analyze spending patterns?

GROUP BY aggregates rows sharing a common value into summary groups:

- GROUP BY category_name with SUM(amount) shows total spending per category.
- GROUP BY user_id with SUM(amount) gives per-user spending totals.
- Used with HAVING, it filters groups — e.g., show only categories above a threshold.
- Combined with MONTH(expense_date), it produces monthly spending breakdowns.

4. Scenario where ROLLBACK is required during expense entry

Scenario: A user submits a payment requiring two steps — inserting into expenses AND updating budget_remaining. The INSERT succeeds but the UPDATE fails due to a server error. Without ROLLBACK, the expense is recorded but the budget is not reduced, causing corrupted financial data.

```
START TRANSACTION;
```

```
INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date)
```

```
VALUES (101, 1, 2, 1500.00, CURDATE());

UPDATE budget SET budget_remaining = budget_remaining - 1500.00
WHERE user_id = 1;

-- If any step fails:
ROLLBACK;

-- If all steps succeed:
COMMIT;
```

5. How do views help users track monthly expenses efficiently?

A VIEW is a saved SELECT query that works like a virtual table:

- Simplification: Pre-joins users, categories, and expenses — one simple SELECT instead of complex JOINS every time.
- Security: Can expose only certain columns, hiding sensitive fields like emails.
- Consistency: All reports use the same logic defined once in the view.
- Reusability: Multiple dashboards query the same view, reducing duplicate code.

6. Why use triggers for automatic category or balance updates?

Triggers are automatic PL/SQL blocks that execute on INSERT, UPDATE, or DELETE events:

- Automation: When an expense is inserted, a trigger can update a monthly_summary table automatically.
- Consistency: Business logic applies to all sources of data entry, not just one application.
- Audit Trail: An AFTER INSERT trigger can log every expense to an audit_log table.
- Validation: A BEFORE DELETE trigger can prevent deletion of categories with linked expenses.

Section B: SQL Hands-On

B1. DDL Understanding

Why Foreign Keys are Used

The user_id in the expenses table is a FOREIGN KEY referencing users(user_id). This means:

- Every expense must belong to an existing user — invalid user_id values are rejected.
- It creates a guaranteed relationship between tables — JOINS always return valid data.
- Integrity is enforced at the database level, not just in application code.

Issue if Foreign Keys were Removed: Orphaned Records

If a user is deleted from users but expenses has no foreign key, all expense rows for that user remain. These become orphaned records — expenses pointing to a non-existent user. JOINS return NULL, SUM queries include unattributable amounts, and reports become unreliable.

B2. DML Operations

Insert 5 Users

```
INSERT INTO users (user_id, name, email, created_at) VALUES
(1, 'Aarav Shah', 'aarav.shah@email.com', '2024-01-10'),
(2, 'Priya Mehta', 'priya.mehta@email.com', '2024-02-15'),
(3, 'Rohan Patel', 'rohan.patel@email.com', '2024-03-20'),
(4, 'Sneha Joshi', 'sneha.joshi@email.com', '2024-04-05'),
(5, 'Karan Desai', 'karan.desai@email.com', '2024-05-01');
```

Insert 3 Categories

```
INSERT INTO categories (category_id, category_name) VALUES
(1, 'Food'),
(2, 'Rent'),
(3, 'Entertainment');
```

Insert 10 Expense Records

```
INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date) VALUES
(1, 1, 1, 450.00, '2024-06-01'),
(2, 1, 2, 8000.00, '2024-06-02'),
(3, 2, 1, 320.50, '2024-06-03'),
(4, 2, 3, 750.00, '2024-06-05'),
(5, 3, 1, 210.00, '2024-06-07'),
(6, 3, 2, 7500.00, '2024-06-08'),
(7, 3, 3, 980.00, '2024-06-10'),
(8, 4, 1, 560.00, '2024-06-12'),
(9, 5, 3, 1200.00, '2024-06-14'),
(10, 5, 2, 9000.00, '2024-06-15');
```

Update an Incorrect Expense

```
UPDATE expenses
SET    amount = 350.00
WHERE  expense_id = 3;
```

Delete an Expense Below Threshold

```
DELETE FROM expenses
WHERE amount < 250.00;
```

B3. Data Retrieval

All Expenses with User and Category Details (INNER JOIN)

```
SELECT
    e.expense_date,
    e.amount,
```

```

        u.name          AS user_name,
        c.category_name
FROM expenses e
    INNER JOIN users      u ON e.user_id    = u.user_id
    INNER JOIN categories c ON e.category_id = c.category_id
ORDER BY e.expense_date;

```

Total Expense Amount per Category

```

SELECT
    c.category_name,
    SUM(e.amount) AS total_spent
FROM expenses e
    INNER JOIN categories c ON e.category_id = c.category_id
GROUP BY c.category_name
ORDER BY total_spent DESC;

```

Users Sorted by Total Spending

```

SELECT
    u.name,
    SUM(e.amount) AS total_spending
FROM expenses e
    INNER JOIN users u ON e.user_id = u.user_id
GROUP BY u.user_id, u.name
ORDER BY total_spending DESC;

```

B4. Views

Create ActiveUsersView

```

CREATE VIEW ActiveUsersView AS
SELECT u.name, u.email
FROM users u
WHERE (
    SELECT COUNT(*) FROM expenses e
    WHERE e.user_id = u.user_id
) > 5;

```

Query the View

```

SELECT * FROM ActiveUsersView;

```

Section C: Mini Project – Expense Tracker DB

1. CRUD Queries

CREATE

```

INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date)
VALUES (11, 2, 1, 480.00, CURDATE());

```

READ

```
SELECT
    e.expense_id,
    e.expense_date,
    u.name          AS user_name,
    c.category_name,
    e.amount
FROM expenses e
    INNER JOIN users      u ON e.user_id      = u.user_id
    INNER JOIN categories c ON e.category_id = c.category_id
WHERE e.user_id = 2
ORDER BY e.expense_date DESC;
```

UPDATE

```
UPDATE expenses
SET     amount = 530.00
WHERE  expense_id = 11;
```

DELETE

```
DELETE FROM expenses
WHERE expense_id = 11;
```

2. Stored Procedure – Monthly User Expense Calculator

```
DELIMITER $$

CREATE PROCEDURE GetMonthlyExpense(
    IN  p_user_id  INT,
    IN  p_month    INT,
    IN  p_year     INT,
    OUT p_total    DECIMAL(10,2)
)
BEGIN
    SELECT COALESCE(SUM(amount), 0.00)
    INTO   p_total
    FROM   expenses
    WHERE  user_id          = p_user_id
          AND MONTH(expense_date) = p_month
          AND YEAR(expense_date)  = p_year;
END $$

DELIMITER ;

-- Calling the procedure:
CALL GetMonthlyExpense(1, 6, 2024, @total);
SELECT @total AS 'Monthly Total Expense';
```

3. COMMIT and ROLLBACK Demonstration

COMMIT permanently saves all changes. ROLLBACK undoes all changes since the last COMMIT.

Successful Transaction – COMMIT

```
START TRANSACTION;

INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date)
VALUES (12, 3, 2, 7800.00, CURDATE());

COMMIT;
```

Failed Transaction – ROLLBACK

```
START TRANSACTION;

DELETE FROM expenses WHERE expense_id = 10;

-- Wrong deletion, undo it
ROLLBACK;

SELECT * FROM expenses WHERE expense_id = 10;
```

SAVEPOINT Example

```
START TRANSACTION;

INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date)
VALUES (13, 4, 3, 600.00, CURDATE());

SAVEPOINT sp1;

INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date)
VALUES (14, 4, 1, 99999.00, CURDATE()); -- incorrect amount

ROLLBACK TO SAVEPOINT sp1;

COMMIT; -- only expense 13 is saved
```

- End of Assessment -